

Telco Churn MCP

A local MCP server that answers natural-language questions about 7,043 telco customers with numbers you can check. Built for Claude Code and Codex. Repo: github.com/ortrsa/cheq

DESIGN: WHAT IT IS AND HOW IT ANSWERS

The core is a **governed text-to-SQL pipeline** behind one tool, `ask_data`. Each stage is a plain function:

```
route → generate SQL → guard (AST) → execute (sandbox) → repair ≤2 → synthesize → ground
```

- **Semantic layer** in the prompt: every column's role and exact values, metric definitions, 8 data traps, 15 verified examples. The model never guesses that "fiber" means `'Fiber Optic'`.
- **Guard** parses SQL with sqlglot: single SELECT only, known tables and columns, no file functions, literals inside the value domain. Failures return a precise repair hint.
- **Sandbox**: DuckDB read-only, external access off, configuration locked, so even a SQL the guard missed cannot write or read files.
- **Grounding**: every number in the narrative must match a result cell, or the narrative is withheld and only the table returns.

Two questions need statistics, not SQL, so they get deterministic tools: `segment_churn` (Wilson intervals, lift, Benjamini–Hochberg q-values, small-segment suppression) and `at_risk_customers` (logistic regression, out-of-fold scores, probability × ARR, up to 3 risk-raising reasons measured against the average customer). Every response is `{result, caveats, meta}` with the SQL, assumptions and token usage.

WHY THIS, AND WHAT I AVOIDED

- **SQL, not RAG.** Retrieval returns similar rows; it cannot count or average 7,043 of them, so it produces confident wrong numbers on exactly the questions users ask.
- **Governed, not raw text-to-SQL.** Raw generation invents columns, misspells values, and reads a 12-customer city as a trend. Each guard targets one of those failures.
- **Measured, not assumed.** 30 held-out questions, ablation from schema-only to full pipeline: the semantic layer lifts traps from 67% to 100%; full pipeline 100%, all attacks blocked. One run, small set: a regression guard, not a benchmark.
- **Leakage treated as a real risk.** `satisfaction_score` is collected at exit and `churn_score` is built from the outcome. Training on them lifts AUC from 0.893 to 0.998, a model that looks excellent and predicts nothing. The firewall blocks both.
- **Avoided:** vector DB, fine-tuning, autonomous agent loops, a semantic answer cache (similar questions can mean different things), and scope I could not defend line by line.

PRODUCTION CHANGES

- Semantic layer generated from the warehouse's dbt models instead of hand-written YAML.
- Remote MCP over streamable HTTP with OAuth, role-based and row-level access.
- Eval suite as a release gate; prompts and models versioned and canaried.
- Verified-query library grown from user feedback: approved answers become examples.
- Tracing and cost dashboards per stage; token budgets and rate limits per user.
- Scheduled retraining with drift monitoring; uplift models trained on real campaign A/B results.

RISKS AND HOW THEY ARE PREVENTED

Risk	Prevention
Hallucinated column or value	Exact value domains in context; AST guard blocks with a nearest-match hint; repair loop
Correct SQL, wrong number in the answer	Grounding check; table-only fallback
Prompt injection or destructive SQL	Router refuses first; guard allows only SELECT; read-only, network-disabled sandbox. Adversarial pass rate: 100%
Misleading analysis	Traps for leaky fields, quarterly rate, small samples, confounded offers; leaky columns flagged in SQL and segments
Misleading model explanations	Redundant features excluded (a test checks full rank); reasons centred on the average customer and risk-raising only. Price is kept by choice, so coefficients read "at the same price"
Ambiguous question	Documented defaults ("current customers" = active); assumptions returned with the answer
"Model score" vs the <code>churn_score</code> column	Router sends questions about the model to its model card, answered without an LLM; questions about the column still go to SQL

BUSINESS IMPACT AT CHEQ

- **CHEQ's own retention.** The same engine on CHEQ's subscription base (plan, modules, usage, support tickets) gives customer success a ranked, explained at-risk list and lets anyone ask "which segments are churning" without waiting on an analyst.
- **A product pattern for customers.** Marketers and security teams ask CHEQ's traffic data questions in plain language: "how much of last month's paid spend went to invalid traffic, by campaign?" For a company whose value is trustworthy data, answers that ship with their SQL, sample sizes and caveats are the differentiator.
- **Agent-ready.** As buyers connect AI agents to their tools, a governed MCP server is the safe way to expose CHEQ intelligence: agents get verified numbers, never raw database access.